When Does Decomposition Pay Off? A Pilot Study of Hierarchical Decomposition and Confidence-Based Routing for Multi-Agent Execution

Yizhang Cai[1]

School of Artificial Intelligence, Beijing University of Posts and Telecommunications, Beijing, China
2025111259@bupt.cn

Abstract. Multi-agent frameworks decompose a request into subtasks and route each subtask to a component that is supposed to handle it better than a single model would. The published frameworks in this space are mostly evaluated on tasks where decomposition is expected to help, so the conditions under which it cannot help are rarely measured. This paper reports a small controlled pilot for a five-role pipeline - planner, confidence router, specialist, aggregator, verifier - against a single-agent baseline on auto-generated arithmetic chains with computed ground truth. On 6-step chains (n=24) both conditions answer every task correctly; on 12-step chains (n=8) both conditions also answer every task correctly. The pipeline spends 5.3 model calls against 1.0 and roughly thirteen times the latency, and the confidence router sends 1.83 of about 2 subtasks to the specialist, so the routing signal carries almost no information on this family. We report the null result, analyse why the family cannot discriminate between the two designs, and state the task properties that a decomposition framework must target before its benefit becomes measurable.

Keywords: multi-agent systems; task decomposition; confidence-based routing; negative result; pilot study

1 Introduction

A single call to a large language model handles many multi-step requests outright. Frameworks that decompose a request across several roles exist because some requests are expected to exceed what one call can do: they need different tools, different context, or enough steps that per-step errors accumulate into a wrong answer [3,4,6,7]. The design argument is sound, but the empirical part of it is usually demonstrated on suites that were chosen because decomposition helps.

This paper asks the complementary question at pilot scale: if the task family leaves no headroom, what does a decomposition pipeline cost, and what does that tell us about when it should be used? We implement a five-role pipeline - planner, confidence router, specialist, aggregator, verifier - and compare it against a single-agent baseline on two auto-generated families of arithmetic chain tasks whose answers are computed in Python rather than authored. Both conditions are run with the same model at temperature 0 on the same questions.

The contributions are deliberately small and factual:

1. A controlled comparison on two chain families (6 steps, n=24; 12 steps, n=8) in which both conditions answer every task correctly.
2. A cost accounting: 5.3 versus 1.0 model calls per task and about thirteen times the latency, with a confidence router that routed 1.83 of about 2 subtasks to the specialist and therefore discriminated almost nothing.
3. A specification of the conditions a task family must satisfy for the decomposition framework to be measurable at all, which is what the null result is actually evidence about.

2 Related Work

Multi-agent frameworks allocate roles to separate model calls: conversational orchestration [6], software-engineering role assignment [7], and model selection across a library of specialist models [3]. Planning work supplies the structure that such frameworks use, either by interleaving reasoning and acting [4] or by searching over partial solutions [5]. Reflection mechanisms complete the loop by letting an agent inspect its own output and revise [2,3]. Our pipeline borrows the planner/executor split from [5], the specialist selection idea from [3], and the revision step from [2], and then measures what remains when the underlying task does not require any of it.

3 Method

The pipeline has five roles, each a separate model call with its own system prompt.

Planner. Decomposes the question into at most three subtasks and returns JSON: an identifier, a kind (`calc`, `lookup`, `verify`), a natural-language description, and a confidence value between 0 and 1 that a general agent can complete the subtask unaided.

Router. Sends a subtask to the specialist when its confidence is below 0.7 or its kind is `calc`; otherwise the subtask goes to a general solver. The threshold is fixed for the whole pilot.

Specialist. A calculator prompt that receives exactly one subtask.

Aggregator. Combines subtask results into a draft answer.

Verifier. Recomputes independently from the original question and the draft, then emits the final number.

The baseline is a single call to the same model with the same question and an instruction to work step by step and end with `ANSWER: <number>`.

4 Setup

Both task families are generated programmatically with a fixed seed; every answer is computed in Python, so the ground truth is exact rather than authored. The 6-step family draws three-digit operands and mixes multiplication, subtraction, addition, and division, asking for a result rounded to two decimals. The 12-step family starts from a four-digit number and applies twelve randomly ordered operations, with operands drawn from two ranges. Answers are compared with a tolerance of `max(0.02, |answer| * 1e-4)`.

Both conditions run `deepseek-v4-flash` at temperature 0 through the chat-completions API, one task per client so that the call accounting is exact. Metrics are answer accuracy, model calls per task, and wall-clock latency per task. Each task is run once; there is no repetition across seeds.

5 Results

Table 1. Accuracy and cost on two generated chain families; one run per condition.

Family	Accuracy, baseline	Accuracy, pipeline	Calls, baseline to pipeline	Latency, baseline to pipeline
6-step chains, n=24	24/24	24/24	1.00 to 5.29	1.4 s to 18.5 s
12-step chains, n=8	8/8	8/8	1.00 to 5.12	9.3 s to 74.2 s

The pipeline answers every task correctly, and so does the baseline. The pipeline buys that outcome with roughly five times as many model calls and, on the long chains, about eight times the latency of the already slower baseline - 74.2 seconds against 9.3. The router sent 1.83 of about 2 subtasks to the specialist, and the verifier had no error to correct in any run: the first pass was already right on all 32 tasks.

6 Why This Family Cannot Discriminate

The comparison is uninformative about the pipeline for a reason that can be stated precisely. Let a task require `k` dependent steps and let each step be right with probability `p`. A single pass is correct with probability about `p^k`, while a pipeline that checks each step changes the exponent. Decomposition wins when `p^k` is small enough to matter, that is, when steps are hard enough or numerous enough for end-to-end accuracy to decay. In this pilot the model answered every step correctly at any chain length we tried, so `p` is close enough to 1 that `p^k` stays near 1 for `k = 12`, and the pipeline has nothing to repair.

Two secondary observations follow, and they are the part of this pilot we consider transferable. First, the confidence signal was uninformative: with mean confidence above the threshold, nearly every subtask was routed to the specialist, so the routing rule behaved as an unconditional branch and its extra call cost bought nothing. A confidence router is only worth its overhead if confidence separates easy from hard subtasks, which is testable before the framework is built. Second, cost scales with the number of roles, not with the difficulty of the question, so a pipeline pays its full price on tasks that need no decomposition - in this pilot, 24 of 24 tasks in the first family.

7 What the Result Implies for Framework Design

The null result locates the requirement rather than dismissing the design. A decomposition pipeline becomes the right tool when at least one of the following holds.

Per-step error accumulates. The family must have a per-step error rate high enough that end-to-end success decays with length, which is what makes a verifier or a repair step worth a call. *Subtasks need different tools or contexts.* Routing pays when two subtasks genuinely require different capabilities, in which case the specialist is not interchangeable with the general solver. *The router's confidence is calibrated.* The routing decision must separate subtasks, otherwise the rule reduces to a constant and the additional call is pure overhead.

Families that plausibly satisfy these conditions are tool-use tasks with failing calls, where the environment returns errors that force a plan revision, and tasks whose steps depend on each other in ways that a one-shot plan cannot enumerate in advance. The failure-repair case is measured in a companion pilot, where the same pipeline structure does separate the two designs.

8 Limitations

This is a pilot: one hosted model, generated arithmetic tasks, a single run per task, and no human review of the intermediate steps. The pipeline implements one particular decomposition prompt and one fixed confidence threshold, so the negative result applies to this configuration on this family and not to decomposition in general. The latency figures come from a shared endpoint and are indicative rather than controlled. The comparison also excludes the setting where decomposition is most often justified in practice - heterogeneous tools with real failure modes - because this pilot deliberately isolates the no-headroom case.

9 Conclusion

On two generated families of arithmetic chains, a five-role decomposition pipeline matched a single-agent baseline exactly at 32 of 32 tasks while spending about five times the model calls and roughly thirteen times the latency, with a confidence router that discriminated almost nothing. The useful output of the pilot is a set of conditions - error accumulation, heterogeneous capability requirements, and a calibrated router - that a task family must satisfy before a decomposition framework can show a measurable benefit.

References

1. A. Madaan et al., "Self-Refine: iterative refinement with self-feedback," in NeurIPS 36, 2023.
2. N. Shinn et al., "Reflexion: language agents with verbal reinforcement learning," in NeurIPS 36, 2023.
3. Y. Shen et al., "HuggingGPT: solving AI tasks with ChatGPT and its friends in Hugging Face," in NeurIPS 36, 2023, arXiv:2303.17580.

4. S. Yao et al., "ReAct: synergizing reasoning and acting in language models," in ICLR, 2023, arXiv:2210.03629.
5. S. Yao et al., "Tree of Thoughts: deliberate problem solving with large language models," in NeurIPS 36, 2023, arXiv:2305.10601.
6. Q. Wu et al., "AutoGen: enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
7. S. Hong et al., "MetaGPT: meta programming for a multi-agent collaborative framework," in ICLR, 2024, arXiv:2308.00352.